

# 新哲文院算法社 2026-2027 学年 Python 教材 · 答案与解析

## 新哲文院算法社 2026-2027 学年 Python 教材 · 答案与解析

本文件对应 `Python教材_新哲文院算法社_2026-2027.md`  
按教材顺序排列，包含课堂练习和课后作业的完整解答与解析

### 第一部分：基础掌握 · 答案

#### 第一单元 · 课堂练习 1.1

题目：编写一个程序，要求用户输入姓名和年龄，然后输出一句完整的介绍。

##### ✓ 解答

Python

```
1 name = input("请输入姓名：")
2 age = input("请输入年龄：")
3 print(f"你好，我叫{name}，今年{age}岁。")
```

##### 📝 解析

- `input()` 永远返回字符串，年龄用 `input()` 接收后直接输出即可
- 如果后续需要做数学运算，才需要用 `int(age)` 转换
- f-string 是 Python 中最优雅的字符串格式化方式

#### 第二单元 · 课堂练习 2.1

练习 1：输入圆的半径，输出面积和周长。

## ✓ 解答

Python

```
1 r = float(input())
2 PI = 3.14159
3 area = PI * r ** 2
4 circumference = 2 * PI * r
5 print(f"面积={area}, 周长={circumference}")
```

## 📝 解析

- 需要 `float()` 转换，因为半径可能是小数
- 题目指定使用 `PI = 3.14159`，因此面积为 `PI * r ** 2`，周长为 `2 * PI * r`

**练习2：**输入一个三位数，输出百位、十位、个位。

## ✓ 解答

Python

```
1 n = int(input())
2 hundreds = n // 100      # 百位：整除100
3 tens = (n // 10) % 10    # 十位：先整除10再取余10
4 ones = n % 10            # 个位：取余10
5
6 print(f"百位={hundreds}, 十位={tens}, 个位={ones}")
```

## 📝 解析

- `//` 是整除运算符，丢弃小数部分
- 十位的核心技巧：`(n // 10) % 10`，先去掉个位再取新数的个位
- 也可以转为字符串：`str(n)[0]` 是百位，`str(n)[1]` 是十位

## 第三单元 · 课堂练习 3.1

**练习1：**判断闰年。

## ✓ 解答

Python

```
1 year = int(input())
2
3 if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
4     print("闰年")
5 else:
6     print("平年")
```

### 解析

- 闰年规则：能被4整除但不能被100整除，**或**能被400整除
- 注意 `and` 和 `or` 的优先级：`and` 高于 `or`，但加括号更清晰
- 测试关键点：2000（闰年）、1900（平年）、2024（闰年）

**练习2：**输入三个整数，找最大值。

### 解答

Python

```
1 a, b, c = map(int, input().split())
2
3 # 方法一：用 max 函数
4 print(f"最大值是{max(a, b, c)}")
5
6 # 方法二：不用内置函数
7 max_val = a
8 if b > max_val:
9     max_val = b
10 if c > max_val:
11     max_val = c
12 print(f"最大值是{max_val}")
```

### 解析

- `map(int, input().split())` 是竞赛中的标准输入写法
- 不用内置函数时，用"假设第一个最大，逐个比较更新"的策略

**练习1：**用 while 循环计算 n 的阶乘。

✅ 解答

Python

```
1 n = int(input())
2 result = 1
3
4 i = 1
5 while i <= n:
6     result *= i
7     i += 1
8
9 print(f"{n}! = {result}")
```

📝 解析

- 阶乘定义： $n! = 1 \times 2 \times 3 \times \dots \times n$
- 初始值 `result = 1`（不是  $0!$ ），`i` 从 1 开始
- while 循环条件 `i <= n`，确保 `n` 也被乘进去

**练习2：**用 for 循环输出三角形图案。

✅ 解答

Python

```
1 n = int(input())
2
3 for i in range(1, n + 1):
4     print("*" * i)
```

📝 解析

- `"*" * i` 是字符串重复操作，生成 `i` 个星号
- `range(1, n+1)` 让 `i` 从 1 到 `n`

**练习3：**100 以内能被 3 整除但不能被 5 整除的数，每行 5 个。

✅ 解答

Python

```
1 count = 0
2 for i in range(1, 101):
3     if i % 3 == 0 and i % 5 != 0:
4         print(i, end=' ')
5         count += 1
6     if count % 5 == 0:
7         print() # 换行
```

#### 解析

- `end=' '` 让 print 不换行，用空格分隔
- `count % 5 == 0` 时换行，实现每行 5 个

## 第四单元 · 课堂练习 4.1

练习 1：输入若干整数，输出和与平均值。

#### 解答

Python

```
1 numbers = list(map(int, input().split()))
2 total = sum(numbers)
3 avg = total / len(numbers)
4 print(f"和={total}, 平均值={avg:.2f}")
```

#### 解析

- `sum()` 和 `len()` 是内置函数，直接求和和计数
- `:.2f` 保留 2 位小数

练习 2：统计字符出现次数。

#### 解答

Python

```
1 text = input()
2 count = {}
3
4 for ch in text:
5     if ch in count:
6         count[ch] += 1
7     else:
8         count[ch] = 1
9
10 for ch, cnt in count.items():
11     print(f"{ch}:{cnt}", end=' ')
```

### 解析

- 字典的键是字符，值是出现次数
- `if ch in count` 判断键是否存在
- 也可以用 `count.get(ch, 0) + 1` 的写法

**练习3：**5个学生的姓名和成绩，输出最高分。

### 解答

Python

```
1 students = {}
2 for i in range(5):
3     name, score = input().split()
4     students[name] = int(score)
5
6 max_name = max(students, key=students.get)
7 print(f"最高分: {max_name}, {students[max_name]}分")
```

### 解析

- `max(dict, key=dict.get)` 返回值最大的键
- 这是字典操作的高级技巧

练习：编写 `is_prime(n)` 函数，找出 2~100 的所有素数。

### ✓ 解答

Python

```
1 import math
2
3 def is_prime(n):
4     """判断 n 是否为素数"""
5     if n < 2:
6         return False
7     for i in range(2, int(math.sqrt(n)) + 1):
8         if n % i == 0:
9             return False
10    return True
11
12 # 找出 2~100 的所有素数
13 primes = []
14 for i in range(2, 101):
15     if is_prime(i):
16         primes.append(i)
17
18 print(*primes)
```

### 📝 解析

- 优化：只需检查到 `sqrt(n)`，如果  $n$  有大于  $\sqrt{n}$  的因子，那必然有小于  $\sqrt{n}$  的对应因子
- `int(math.sqrt(n)) + 1` 确保不漏检

---

## 第六单元 • 课堂练习 6.1

练习：文件读写与异常处理。

### ✓ 解答

Python

```
1 filename = input("请输入文件名: ")
2
3 try:
4     with open(filename, "r", encoding="utf-8") as f:
5         lines = f.readlines()
6         char_count = sum(len(line) for line in lines)
7         print(f"行数: {len(lines)}")
8         print(f"字符数: {char_count}")
9 except FileNotFoundError:
10    print(f"文件 '{filename}' 不存在, 正在创建...")
11    with open(filename, "w", encoding="utf-8") as f:
12        f.write("这是新创建的文件\n")
13    print("文件已创建。")
```

### 解析

- `try-except` 捕获 `FileNotFoundError` 异常
- `readlines()` 返回行列表, `len()` 即得行数
- 生成器表达式 `sum(len(line) for line in lines)` 高效统计字符

---

## 第一部分：基础掌握 · 课后作业答案

---

### 作业 1：温度转换器

#### 解答

Python

```
1 celsius = float(input("请输入摄氏温度: "))
2 fahrenheit = celsius * 9 / 5 + 32
3 kelvin = celsius + 273.15
4
5 print(f"华氏温度: {fahrenheit:.2f}°F")
6 print(f"开尔文温度: {kelvin:.2f}K")
```



### 解析

- 公式记忆:  $F = C \times 9/5 + 32$ ,  $K = C + 273.15$
  - `:.2f` 格式化输出, 保留 2 位小数
- 

## 作业 2: 简易计算器

### 解答

Python

```
1 a = float(input("请输入第一个数: "))
2 b = float(input("请输入第二个数: "))
3 op = input("请输入运算符(+, -, *, /): ")
4
5 if op == '+':
6     print(f"{a} + {b} = {a + b}")
7 elif op == '-':
8     print(f"{a} - {b} = {a - b}")
9 elif op == '*':
10    print(f"{a} * {b} = {a * b}")
11 elif op == '/':
12    try:
13        result = a / b
14        print(f"{a} / {b} = {result}")
15    except ZeroDivisionError:
16        print("错误: 除数不能为零!")
17 else:
18    print("不支持的运算符!")
```

### 解析

- 用 `try-except` 处理除零错误
  - `elif` 链处理不同运算符
  - 输入验证可以用 `if op not in '+-*/'`
- 

## 作业 3: 回文判断

## ✅ 解答

Python

```
1 s = input("请输入字符串: ")
2
3 # 方法一: 切片反转
4 if s == s[::-1]:
5     print("是回文")
6 else:
7     print("不是回文")
8
9 # 方法二: 双指针
10 left, right = 0, len(s) - 1
11 is_palindrome = True
12 while left < right:
13     if s[left] != s[right]:
14         is_palindrome = False
15         break
16     left += 1
17     right -= 1
18
19 print("是回文" if is_palindrome else "不是回文")
```

## 📝 解析

- `s[::-1]` 是 Python 中最简洁的翻转字符串方法
- 双指针法更通用, 适合其他语言

---

## 作业 4: 成绩统计

## ✅ 解答

## Python

```
1 scores = []
2 for i in range(10):
3     scores.append(int(input(f"请输入第{i+1}个成绩: ")))
4
5 max_score = max(scores)
6 min_score = min(scores)
7 avg_score = sum(scores) / len(scores)
8 pass_count = sum(1 for s in scores if s >= 60)
9 pass_rate = pass_count / len(scores) * 100
10
11 print(f"最高分: {max_score}")
12 print(f"最低分: {min_score}")
13 print(f"平均分: {avg_score:.1f}")
14 print(f"及格率: {pass_rate:.1f}%")
```

### 解析

- 题目要求固定输入 10 个成绩，因此循环次数直接写为 `10`
- `sum(1 for s in scores if s >= 60)` 是生成器表达式统计满足条件的个数
- 也可以用列表推导式 + `len()`

## 作业 5：手写冒泡排序

### 解答

Python

```
1 def bubble_sort(lst):
2     """冒泡排序（升序）"""
3     n = len(lst)
4     for i in range(n - 1):
5         for j in range(n - 1 - i):
6             if lst[j] > lst[j + 1]:
7                 lst[j], lst[j + 1] = lst[j + 1], lst[j]
8     return lst
9
10 # 测试
11 nums = list(map(int, input().split()))
12 bubble_sort(nums)
13 print(*nums)
```

### 解析

- 外层循环 `n-1` 轮，因为 `n` 个元素最多需要 `n-1` 轮
- 内层循环 `n-1-i` 次，每轮后最大的 `i` 个已就位
- Python 元组解包 `a, b = b, a` 实现无临时变量交换

---

## 第二部分：竞赛进阶 · 答案

---

### 第五单元 · 课堂练习 5.1

练习 1：用冒泡排序对字符串按长度排序。

### 解答

Python

```
1 words = input().split()
2
3 # 按字符串长度排序（冒泡）
4 n = len(words)
5 for i in range(n - 1):
6     for j in range(n - 1 - i):
7         if len(words[j]) > len(words[j + 1]):
8             words[j], words[j + 1] = words[j + 1], words[j]
9
10 print(*words)
```

### 解析

- 比较条件从 `>` 改为 `len() > len()`
- 冒泡排序的核心结构不变，只改比较逻辑

**练习2：**桶排序统计 0~9 出现次数。

### 解答

Python

```
1 import random
2
3 # 生成10000000个0~9的随机数（模拟输入）
4 data = [random.randint(0, 9) for _ in range(1000000)]
5
6 # 桶排序/计数
7 buckets = [0] * 10
8 for num in data:
9     buckets[num] += 1
10
11 for i in range(10):
12     print(f"数字{i}: {buckets[i]}次")
```

### 解析

- 桶排序特别适合数据范围小、数据量大的场景
  - 时间复杂度  $O(n+k)$ ，比冒泡的  $O(n^2)$  快得多
-

## 第五单元 · 课后作业 5

作业 1：冒泡排序降序。

✓ 解答

Python

```
1 def bubble_sort_desc(lst):
2     """冒泡排序（降序）"""
3     n = len(lst)
4     for i in range(n - 1):
5         for j in range(n - 1 - i):
6             if lst[j] < lst[j + 1]:      # 注意：小于号（升序是大于号）
7                 lst[j], lst[j + 1] = lst[j + 1], lst[j]
8     return lst
9
10 nums = list(map(int, input().split()))
11 bubble_sort_desc(nums)
12 print(*nums)
```

✎ 解析

- 降序只需把比较符号从 `>` 改为 `<`
- 冒泡方向不变，只是"轻的往上冒"变成"重的往上冒"

作业 2：按成绩降序输出学生姓名。

✓ 解答

## Python

```
1 n = int(input())
2 students = []
3
4 for i in range(n):
5     name, score = input().split()
6     students.append((name, int(score)))
7
8 # 桶排序：成绩作为桶索引
9 buckets = [[] for _ in range(101)]
10
11 for name, score in students:
12     buckets[score].append(name)
13
14 # 从高分到低分输出
15 for score in range(100, -1, -1):
16     if buckets[score]:
17         for name in buckets[score]:
18             print(name, end=' ')
```

### 解析

- 每个桶存一个列表（因为同分可能有多个学生）
- 从 100 遍历到 0 实现降序
- 同分学生的顺序保持输入顺序（因为按顺序 append）

---

## 第六单元 • 课堂练习 6.1

练习 1：全排列中找和为 n 的组合。

### 解答

## Python

```
1 def dfs(start, path, target):
2     """从start开始搜索, path为当前组合"""
3     if sum(path) == target:
4         print(*path)
5         return
6     if sum(path) > target:
7         return
8
9     for i in range(start, target + 1):
10         path.append(i)
11         dfs(i + 1, path, target)    # i+1 保证不重复使用
12         path.pop()
13
14 n = int(input())
15 dfs(1, [], n)
```

### 解析

- `start` 参数保证不重复选同一个数
- 剪枝：当前和已超过目标，直接返回
- 输出顺序天然从小到大

**练习2：**N 皇后问题。

### 解答



```

1 def solve_n_queens(n):
2     """输出N皇后的一种方案"""
3     cols = [False] * n
4     diag1 = [False] * (2 * n - 1) # 主对角线
5     diag2 = [False] * (2 * n - 1) # 副对角线
6     board = [['.'] * n for _ in range(n)]
7
8     def dfs(row):
9         if row == n:
10             for r in board:
11                 print(' '.join(r))
12             print()
13             return True # 只找一种方案
14
15         for col in range(n):
16             d1 = row + col
17             d2 = row - col + n - 1
18             if not cols[col] and not diag1[d1] and not diag2[d2]:
19                 cols[col] = diag1[d1] = diag2[d2] = True
20                 board[row][col] = 'Q'
21
22                 if dfs(row + 1):
23                     return True
24
25             # 回溯
26             board[row][col] = '.'
27             cols[col] = diag1[d1] = diag2[d2] = False
28
29         return False
30
31     dfs(0)
32
33 n = int(input())
34 solve_n_queens(n)

```

### 解析

- 三个布尔数组快速判断列和对角线冲突
- 主对角线编号: `row + col` (值相同的在同一对角线)
- 副对角线编号: `row - col + n - 1` (偏移避免负数)
- 找到一种方案后立即返回

---

## 第六单元 · 课后作业 6

作业 1：岛屿数量（DFS）。

 解答

```
1 def count_islands(grid):
2     """统计岛屿数量"""
3     if not grid:
4         return 0
5
6     rows, cols = len(grid), len(grid[0])
7     visited = [[False] * cols for _ in range(rows)]
8     count = 0
9
10    def dfs(r, c):
11        if (r < 0 or r >= rows or c < 0 or c >= cols
12            or grid[r][c] == 0 or visited[r][c]):
13            return
14        visited[r][c] = True
15        # 四个方向
16        dfs(r-1, c)
17        dfs(r+1, c)
18        dfs(r, c-1)
19        dfs(r, c+1)
20
21    for i in range(rows):
22        for j in range(cols):
23            if grid[i][j] == 1 and not visited[i][j]:
24                count += 1
25                dfs(i, j)
26
27    return count
28
29 # 输入
30 n, m = map(int, input().split())
31 grid = []
32 for i in range(n):
33     grid.append(list(map(int, input().split())))
34
35 print(count_islands(grid))
```

## 解析

- 遍历每个格子，遇到未访问的陆地就 DFS 标记整个岛屿
- 每次启动 DFS 就发现一个新岛屿，count++
- 时间复杂度  $O(n \times m)$ ，每个格子最多访问一次

## 作业2：图中所有路径（DFS）。

### ✅ 解答

Python

```
1 def find_all_paths(graph, start, end):
2     """找出图中从start到end的所有路径"""
3     all_paths = []
4
5     def dfs(current, path):
6         path.append(current)
7
8         if current == end:
9             all_paths.append(path.copy())
10        else:
11            for neighbor in graph.get(current, []):
12                if neighbor not in path: # 避免环
13                    dfs(neighbor, path)
14
15        path.pop() # 回溯
16
17    dfs(start, [])
18    return all_paths
19
20 # 示例图（邻接表）
21 graph = {
22     'A': ['B', 'C'],
23     'B': ['A', 'D', 'E'],
24     'C': ['A', 'F'],
25     'D': ['B'],
26     'E': ['B', 'F'],
27     'F': ['C', 'E']
28 }
29
30 paths = find_all_paths(graph, 'A', 'F')
31 for p in paths:
32     print(" → ".join(p))
```

### 📝 解析

- `neighbor not in path` 防止在环中无限递归
- `path.copy()` 保存当前路径的副本

- 回溯时 `path.pop()` 撤销最后一步

## 第七单元 · 课堂练习 7.1

练习1：马走日（BFS 最短路径）。

### ✅ 解答

Python

```
1 from collections import deque
2
3 def knight_bfs(start, end, n=8):
4     """马从start到end的最少步数"""
5     queue = deque([(start[0], start[1], 0)])
6     visited = {start}
7
8     # 马走日的8个方向
9     moves = [
10         (2, 1), (2, -1), (-2, 1), (-2, -1),
11         (1, 2), (1, -2), (-1, 2), (-1, -2)
12     ]
13
14     while queue:
15         x, y, steps = queue.popleft()
16
17         if (x, y) == end:
18             return steps
19
20         for dx, dy in moves:
21             nx, ny = x + dx, y + dy
22             if 0 <= nx < n and 0 <= ny < n and (nx, ny) not in visited:
23                 visited.add((nx, ny))
24                 queue.append((nx, ny, steps + 1))
25
26     return -1
27
28 print(knight_bfs((0, 0), (7, 7))) # 输出: 6
```

### 📝 解析

- BFS 保证找到的是最短路径

- 马的8个移动方向要写全
- 用集合 `visited` 记录已访问位置

**练习2：**单词接龙（BFS）。

### ✅ 解答

Python

```
1 from collections import deque
2
3 def word_ladder(begin, end, word_list):
4     """从begin到end的最短变换序列长度"""
5     word_set = set(word_list)
6     if end not in word_set:
7         return 0
8
9     queue = deque([(begin, 1)])
10    visited = {begin}
11
12    while queue:
13        word, length = queue.popleft()
14
15        # 尝试变换每个位置的字符
16        for i in range(len(word)):
17            for c in 'abcdefghijklmnopqrstuvwxyz':
18                new_word = word[:i] + c + word[i+1:]
19                if new_word == end:
20                    return length + 1
21                if new_word in word_set and new_word not in visited:
22                    visited.add(new_word)
23                    queue.append((new_word, length + 1))
24
25    return 0
26
27 begin = "hit"
28 end = "cog"
29 word_list = ["hot", "dot", "dog", "lot", "log", "cog"]
30 print(word_ladder(begin, end, word_list)) # 输出: 5
```

### ✎ 解析

- 每次变换一个字符，检查是否在新字典中

- BFS 层级即为变换步数
  - 这是经典 LeetCode 题 (Word Ladder)
- 

## 第七单元 • 课后作业 7

作业 1: 网格最短路径 (BFS)。

 解答

```
1 from collections import deque
2
3 def shortest_path(grid):
4     """BFS 找最短路径"""
5     n, m = len(grid), len(grid[0])
6
7     if grid[0][0] == 1 or grid[n-1][m-1] == 1:
8         return -1
9
10    queue = deque([(0, 0, 0)])
11    visited = [[False] * m for _ in range(n)]
12    visited[0][0] = True
13
14    directions = [(-1, 0), (0, 1), (1, 0), (0, -1)]
15
16    while queue:
17        x, y, steps = queue.popleft()
18
19        if x == n - 1 and y == m - 1:
20            return steps
21
22        for dx, dy in directions:
23            nx, ny = x + dx, y + dy
24            if (0 <= nx < n and 0 <= ny < m
25                and grid[nx][ny] == 0 and not visited[nx][ny]):
26                visited[nx][ny] = True
27                queue.append((nx, ny, steps + 1))
28
29    return -1
30
31 # 输入
32 n, m = map(int, input().split())
33 grid = []
34 for i in range(n):
35     grid.append(list(map(int, input().split())))
36
37 print(shortest_path(grid))
```

## 解析

- 与教材中的 BFS 模板基本一致



- 注意起点或终点本身是障碍物的情况
- BFS 第一次到达终点就是最短路径

**作业 2：**水壶问题。

 **解答**

```
1 from collections import deque
2
3 def can_measure(a, b, c):
4     """判断能否用容量为a和b的水壶量出c升水"""
5     if c > a + b:
6         return False
7
8     queue = deque([(0, 0)])          # (壶A水量, 壶B水量)
9     visited = {(0, 0)}
10
11     while queue:
12         x, y = queue.popleft()
13
14         # 检查是否量出 c 升
15         if x == c or y == c or x + y == c:
16             return True
17
18         # 6种操作
19         states = [
20             (a, y),          # 装满A
21             (x, b),          # 装满B
22             (0, y),          # 倒空A
23             (x, 0),          # 倒空B
24             (min(a, x + y), max(0, x + y - a)), # B倒给A
25             (max(0, x + y - b), min(b, x + y)), # A倒给B
26         ]
27
28         for state in states:
29             if state not in visited:
30                 visited.add(state)
31                 queue.append(state)
32
33     return False
34
35 a, b, c = map(int, input().split())
36 print("可以" if can_measure(a, b, c) else "不可以")
```

### 解析

- 状态 = (壶A水量, 壶B水量)
- 6种操作：装满A、装满B、倒空A、倒空B、A→B、B→A

- BFS 搜索所有可达状态

## 第八单元 · 课堂练习 8.1

练习1：递归求阶乘。

### ✓ 解答

Python

```
1 def factorial(n):
2     """递归求阶乘"""
3     if n <= 1:                # 终止条件
4         return 1
5     return n * factorial(n - 1) # 递归调用
6
7 print(factorial(5))           # 120
```

### ✎ 解析

- 终止条件 `n <= 1` 很重要，否则无限递归
- `factorial(5) = 5 × factorial(4) = 5 × 4 × 3 × 2 × 1`

练习2：递归反转字符串。

### ✓ 解答

Python

```
1 def reverse_string(s):
2     """递归反转字符串"""
3     if len(s) <= 1:          # 终止条件
4         return s
5     return s[-1] + reverse_string(s[:-1])
6
7 print(reverse_string("hello")) # olleh
```

### ✎ 解析

- 把最后一个字符放到最前面，递归处理剩余部分
- `s[:-1]` 是去掉最后一个字符的切片

### 练习3：青蛙跳台阶。

#### ✅ 解答

Python

```
1 def frog_jump(n):
2     """青蛙跳台阶 - 递归版"""
3     if n <= 2:
4         return n
5     return frog_jump(n - 1) + frog_jump(n - 2)
6
7 # 测试
8 for i in range(1, 6):
9     print(f"{i}级: {frog_jump(i)}种")
10
11 # 输出: 1, 2, 3, 5, 8 (斐波那契数列!)
```

#### 📝 解析

- 跳到  $n$  级 = 从  $n-1$  跳 1 步 + 从  $n-2$  跳 2 步
- 结果就是斐波那契数列
- ⚠️ 纯递归效率低（重复计算），实际应用应加记忆化或改为递推

## 第八单元 · 课后作业 8

### 作业1：递归求最大公约数。

#### ✅ 解答

Python

```
1 def gcd(a, b):
2     """辗转相除法 - 递归版"""
3     if b == 0:
4         return a
5     return gcd(b, a % b)
6
7 print(gcd(48, 18))           # 6
8 print(gcd(100, 75))         # 25
```

### 解析

- 核心公式: `gcd(a, b) = gcd(b, a % b)`
- 当 `b = 0` 时, `a` 就是最大公约数
- 这是欧几里得算法, 2300 年前就发明了!

作业 2: 递归判断回文。

### 解答

Python

```
1 def is_palindrome(s):
2     """递归判断回文"""
3     # 去掉空格, 转小写 (可选)
4     s = s.replace(' ', '').lower()
5
6     def check(left, right):
7         if left >= right:
8             return True
9         if s[left] != s[right]:
10            return False
11        return check(left + 1, right - 1)
12
13    return check(0, len(s) - 1)
14
15 print(is_palindrome("level"))          # True
16 print(is_palindrome("hello"))          # False
17 print(is_palindrome("A man a plan a canal Panama")) # True
```

### 解析

- 双指针从两端向中间靠拢
- 内部函数 `check` 接收索引参数
- 处理空格和大小写让判断更鲁棒

作业 3: 子集生成。

### 解答

Python

```
1 def generate_subsets(nums):
2     """生成所有子集"""
3     result = []
4
5     def dfs(index, path):
6         # 每次递归都记录当前路径（包括空集）
7         result.append(path.copy())
8
9         for i in range(index, len(nums)):
10             path.append(nums[i])
11             dfs(i + 1, path)
12             path.pop()
13
14     dfs(0, [])
15     return result
16
17 # 测试
18 nums = [1, 2, 3]
19 subsets = generate_subsets(nums)
20 for s in subsets:
21     print(s)
22
23 # 输出:
24 # []
25 # [1]
26 # [1, 2]
27 # [1, 2, 3]
28 # [1, 3]
29 # [2]
30 # [2, 3]
31 # [3]
```

### 解析

- 每个元素有"选"或"不选"两种状态
- DFS 中 `index` 保证不回头选已跳过的元素
- 子集总数 =  $2^n$ （包括空集）

### 练习1：递推求 $1!+2!+\dots+n!$ 。

#### ✓ 解答

Python

```
1 n = int(input())
2
3 total = 0
4 fact = 1 # 当前阶乘值
5
6 for i in range(1, n + 1):
7     fact *= i          #  $i! = (i-1)! * i$ 
8     total += fact      # 累加到总和
9
10 print(total)
```

#### 📝 解析

- 递推技巧：利用  $i! = (i-1)! \times i$ ，避免重复计算
- 一次循环同时算阶乘和累加

### 练习2：约瑟夫环。

#### ✓ 解答

Python

```
1 def josephus(n, k=3):
2     """约瑟夫环：n个人，数到k出局"""
3     people = list(range(1, n + 1))
4     index = 0
5
6     while len(people) > 1:
7         index = (index + k - 1) % len(people)
8         out = people.pop(index)
9
10    return people[0]
11
12 print(josephus(10))    # 4
13 print(josephus(7))    # 4
14 print(josephus(5))    # 4
```

### 解析

- 用列表模拟圆圈，`pop` 出局的人
  - `(index + k - 1) % len(people)` 计算下一个出局位置
  - `k-1` 是因为从当前人开始数（当前人算第 1 个）
- 

## 第九单元 · 课后作业 9

作业 1：杨辉三角。

### 解答

Python

```
1 def pascals_triangle(n):
2     """生成杨辉三角前n行"""
3     triangle = []
4
5     for i in range(n):
6         row = [1] * (i + 1)      # 每行首尾都是1
7         for j in range(1, i):
8             row[j] = triangle[i-1][j-1] + triangle[i-1][j]
9         triangle.append(row)
10
11     return triangle
12
13 n = int(input())
14 triangle = pascals_triangle(n)
15 for row in triangle:
16     print(*row)
```

### 解析

- 递推公式： $C(n, k) = C(n-1, k-1) + C(n-1, k)$
- 每个数等于上方两数之和
- 首尾元素都是 1

作业 2：铺砖问题。

### 解答



Python

```
1 def pave_ways(n):
2     """1×n 地面用 1×1 和 1×2 铺砖的方法数"""
3     if n <= 2:
4         return n
5
6     # dp[i] = 铺满 1×i 的方法数
7     dp = [0] * (n + 1)
8     dp[1] = 1 # 1×1: 只有1种
9     dp[2] = 2 # 1×2: 两个1×1 或 一个1×2
10
11     for i in range(3, n + 1):
12         dp[i] = dp[i-1] + dp[i-2]
13         # 最后放1×1砖 → dp[i-1]
14         # 最后放1×2砖 → dp[i-2]
15
16     return dp[n]
17
18 for i in range(1, 8):
19     print(f"{i}格: {pave_ways(i)}种")
20
21 # 输出: 1, 2, 3, 5, 8, 13, 21 (又是斐波那契!)
```

### 解析

- 这也是斐波那契数列的变体
- 最后一步要么是 1×1 砖，要么是 1×2 砖
- 递推关系:  $dp[i] = dp[i-1] + dp[i-2]$

**作业3:** 辗转相除法 vs 更相减损术。

### 解答

## Python

```
1 import time
2
3 # 方法一：辗转相除法（取余）
4 def gcd_mod(a, b):
5     while b != 0:
6         a, b = b, a % b
7     return a
8
9 # 方法二：更相减损术（相减）
10 def gcd_sub(a, b):
11     while a != b:
12         if a > b:
13             a -= b
14         else:
15             b -= a
16     return a
17
18 # 效率对比
19 test_cases = [(48, 18), (12345, 6789), (999999, 13), (1000000, 999999)]
20
21 print("测试用例\t取余法\t相减法")
22 for a, b in test_cases:
23     # 取余法
24     t1 = time.perf_counter()
25     for _ in range(10000):
26         gcd_mod(a, b)
27     t_mod = (time.perf_counter() - t1) * 1000
28
29     # 相减法
30     t2 = time.perf_counter()
31     for _ in range(10000):
32         gcd_sub(a, b)
33     t_sub = (time.perf_counter() - t2) * 1000
34
35     print(f"({a},{b})\t{t_mod:.3f}ms\t{t_sub:.3f}ms")
36
37 # 结论：取余法在大多数情况下更快
38 # 但当两数相差悬殊时，相减法可能退化（如 (1000000, 1)）
```

- 辗转相除法（欧几里得）：用取余运算，收敛快
- 更相减损术（中国古代算法）：用减法，直观但可能慢
- 极端情况：gcd(1000000, 1)，减法要减 100 万次，取余只需 1 次

---

## 竞赛进阶 · 综合练习答案

---

### 综合题 1：网格路径数

#### ✓ 解答

Python

```
1 def unique_paths(m, n):
2     """m×n 网格从左上到右下的路径数"""
3     # dp[i][j] = 到达(i,j)的路径数
4     dp = [[1] * n for _ in range(m)]
5
6     for i in range(1, m):
7         for j in range(1, n):
8             dp[i][j] = dp[i-1][j] + dp[i][j-1]
9
10    return dp[m-1][n-1]
11
12 # 也可以用数学公式：C(m+n-2, m-1)
13 from math import comb
14 def unique_paths_math(m, n):
15     return comb(m + n - 2, m - 1)
16
17 print(unique_paths(3, 3))           # 6
18 print(unique_paths_math(3, 7))     # 28
```

#### 解析

- 每个格子只能从上方或左方到达
  - 组合数学解法：总共走 (m-1)+(n-1) 步，选 (m-1) 步向下
  - 时间复杂度  $O(m \times n)$ ，数学公式  $O(\min(m, n))$
-

## 综合题 2：N 皇后所有方案数

### ✓ 解答

Python

```
1 def total_n_queens(n):
2     """N皇后问题：返回所有可行方案的数量"""
3     cols = [False] * n
4     diag1 = [False] * (2 * n - 1)
5     diag2 = [False] * (2 * n - 1)
6     count = 0
7
8     def dfs(row):
9         nonlocal count
10        if row == n:
11            count += 1
12            return
13
14        for col in range(n):
15            d1 = row + col
16            d2 = row - col + n - 1
17            if not cols[col] and not diag1[d1] and not diag2[d2]:
18                cols[col] = diag1[d1] = diag2[d2] = True
19                dfs(row + 1)
20                cols[col] = diag1[d1] = diag2[d2] = False
21
22        dfs(0)
23    return count
24
25 for n in range(1, 9):
26     print(f"{n}皇后: {total_n_queens(n)}种方案")
27
28 # 1:1, 2:0, 3:0, 4:2, 5:10, 6:4, 7:40, 8:92
```

### 解析

- 与课堂练习的 N 皇后类似，但统计所有方案数
- `nonlocal count` 在嵌套函数中修改外部变量
- 8 皇后问题有 92 种不同方案

### 综合题 3：图连通性判断（BFS）

✓ 解答

```
1 from collections import deque
2
3 def is_connected(matrix):
4     """使用邻接矩阵判断无向图是否连通"""
5     n = len(matrix)
6     if n <= 1:
7         return True
8
9     visited = [False] * n
10    queue = deque([0])
11    visited[0] = True
12    count = 1
13
14    while queue:
15        node = queue.popleft()
16        for neighbor, has_edge in enumerate(matrix[node]):
17            if has_edge and not visited[neighbor]:
18                visited[neighbor] = True
19                count += 1
20                queue.append(neighbor)
21
22    return count == n # 所有点都被访问到
23
24 # 测试
25 # 连通图
26 matrix1 = [
27     [0, 1, 1],
28     [1, 0, 1],
29     [1, 1, 0]
30 ]
31 print(is_connected(matrix1)) # True
32
33 # 不连通图
34 matrix2 = [
35     [0, 1, 0, 0],
36     [1, 0, 0, 0],
37     [0, 0, 0, 1],
38     [0, 0, 1, 0]
39 ]
40 print(is_connected(matrix2)) # False
```

## 解析

- 邻接矩阵中 `matrix[u][v]` 非零表示顶点 `u` 和 `v` 之间有边
  - 从任意点（如 0 号）出发 BFS，用 `enumerate(matrix[node])` 找出当前顶点的所有邻接点
  - 如果访问到的节点数等于总节点数，则图连通
  - 这是判断图连通性的标准方法
- 

— 答案与解析文件结束 —

新哲文院算法社 2026-2027 学年